

第5章 数组和广义表

主讲教师：黄绍辉



数组是一种人们非常熟悉的数据结构，几乎所有的程序设计语言都支持这种数据结构或将这种数据结构设定为语言的固有类型。**数组**这种数据结构可以看成**是线性表的推广**。

科学计算中涉及到大量的矩阵问题，在程序设计语言中一般都采用数组来存储，被描述成一个二维数组。但当**矩阵规模很大且具有特殊结构**(对角矩阵、三角矩阵、对称矩阵、稀疏矩阵等)，为减少程序的时间和空间需求，采用自定义的描述方式。

广义表是另一种推广形式的线性表，是一种灵活的数据结构，在许多方面有广泛的应用。



5.1 数组的定义

数组是一组偶对(下标值, 数据元素值)的集合。在数组中, 对于一组有意义的下标, 都存在一个与其对应的值。一维数组对应着一个下标值, 二维数组对应着两个下标值, 如此类推。

数组是由 $n(n > 1)$ 个具有相同数据类型的数据元素 $a_1, a_2, \dots, a_n$ 组成的有序序列, 且该序列必须存储在一块地址连续的存储单元中。

- 数组中的数据元素具有相同数据类型。
- 数组是一种随机存取结构, 给定一组下标, 就可以访问与其对应的数据元素。
- 数组中的数据元素个数是固定的。



数组的抽象数据类型定义

ADT Array{

数据对象: $j_i = 0, 1, \dots, b_i - 1, 1, 2, \dots, n$;

$D = \{ a_{j_1 j_2 \dots j_n} \mid n > 0 \text{ 称为数组的维数, } b_i \text{ 是数组第 } i \text{ 维的长度, } j_i \text{ 是数组元素第 } i \text{ 维的下标, } a_{j_1 j_2 \dots j_n} \in \text{ElemSet} \}$

数据关系: $R = \{R_1, R_2, \dots, R_n\}$

$R_i = \{ \langle a_{j_1 j_2 \dots j_i \dots j_n}, a_{j_1 j_2 \dots j_{i+1} \dots j_n} \rangle \mid 0 \leq j_k \leq b_k - 1, 1 \leq k \leq n \text{ 且 } k \neq i, 0 \leq j_i \leq b_i - 2, a_{j_1 j_2 \dots j_{i+1} \dots j_n} \in D \}$

基本操作:

} ADT Array



由上述定义知， n 维数组中有 $b_1 \times b_2 \times \dots \times b_n$ 个数据元素，每个数据元素都受到 n 维关系的约束。

以二维数组为例讨论。将二维数组看成是一个定长的线性表，其每个元素又是一个定长的线性表。

设二维数组 $A = (a_{ij})_{m \times n}$ ，则

$$A = (\alpha_0, \alpha_1, \dots, \alpha_p) \quad (p = m-1 \text{ 或 } n-1)$$

其中每个数据元素 α_j 是一个列向量(线性表)：

$$\alpha_j = (a_{0j}, a_{1j}, \dots, a_{m-1,j}) \quad 0 \leq j \leq n-1$$

或是一个行向量：

$$\alpha_i = (a_{i0}, a_{i1}, \dots, a_{i,n-1}) \quad 0 \leq i \leq m-1$$

二维数组示例

$$A = \begin{pmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \cdots & \cdots & \cdots & \cdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{pmatrix}$$

(a) 矩阵表示形式

$$A = \begin{pmatrix} \begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \end{bmatrix} \\ \begin{bmatrix} a_{10} & a_{11} & \cdots & a_{1,n-1} \end{bmatrix} \\ \cdots \cdots \cdots \cdots \cdots \\ \begin{bmatrix} a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix} \end{pmatrix}$$

(b) 行向量的一维数组形式

$$A = \left(\begin{pmatrix} a_{00} \\ a_{01} \\ \vdots \\ a_{m-1,1} \end{pmatrix} \begin{pmatrix} a_{01} \\ a_{11} \\ \vdots \\ a_{m-1,1} \end{pmatrix} \mid \begin{pmatrix} a_{0,n-1} \\ a_{1,n-1} \\ \vdots \\ a_{m-1,n-1} \end{pmatrix} \right)$$

(c) 列向量的一维数组形式

5.2 数组的顺序表示和实现

数组一般不做插入和删除操作，也就是说，数组一旦建立，结构中的元素个数和元素间的关系就不再发生变化。因此，一般都是采用顺序存储的方法来表示数组。

计算机的内存结构是一维(线性)地址结构，对于多维数组，将其存放(映射)到内存一维结构时，有个次序约定问题。即必须按某种次序将数组元素排成一系列列，然后将这个线性序列存放到内存中。

二维数组是最简单的多维数组，以此为例说明多维数组存放(映射)到内存一维结构时的次序约定问题。



通常有两种顺序存储方式

(1) 行优先顺序(Row Major Order)：将数组元素按行排列，第 $i+1$ 个行向量紧接在第 i 个行向量后面。对二维数组，按行优先顺序存储的线性序列为：

$a_{00}, a_{01}, \dots, a_{0,n-1}, a_{10}, a_{11}, \dots, a_{1,n-1}, \dots, a_{m-1,0}, a_{m-1,1}, \dots, a_{m-1,n-1}$

PASCAL、C是按行优先顺序存储的，如下图(b)。

(2) 列优先顺序(Column Major Order)：将数组元素按列向量排列，第 $j+1$ 个列向量紧接在第 j 个列向量之后，对二维数组，按列优先顺序存储的线性序列为：

$a_{00}, a_{10}, \dots, a_{m-1,0}, a_{01}, a_{11}, \dots, a_{m-1,1}, \dots, a_{n-1,0}, a_{n-1,1}, \dots, a_{n-1,m-1}$

FORTRAN是按列优先顺序存储的，如下图(c)。

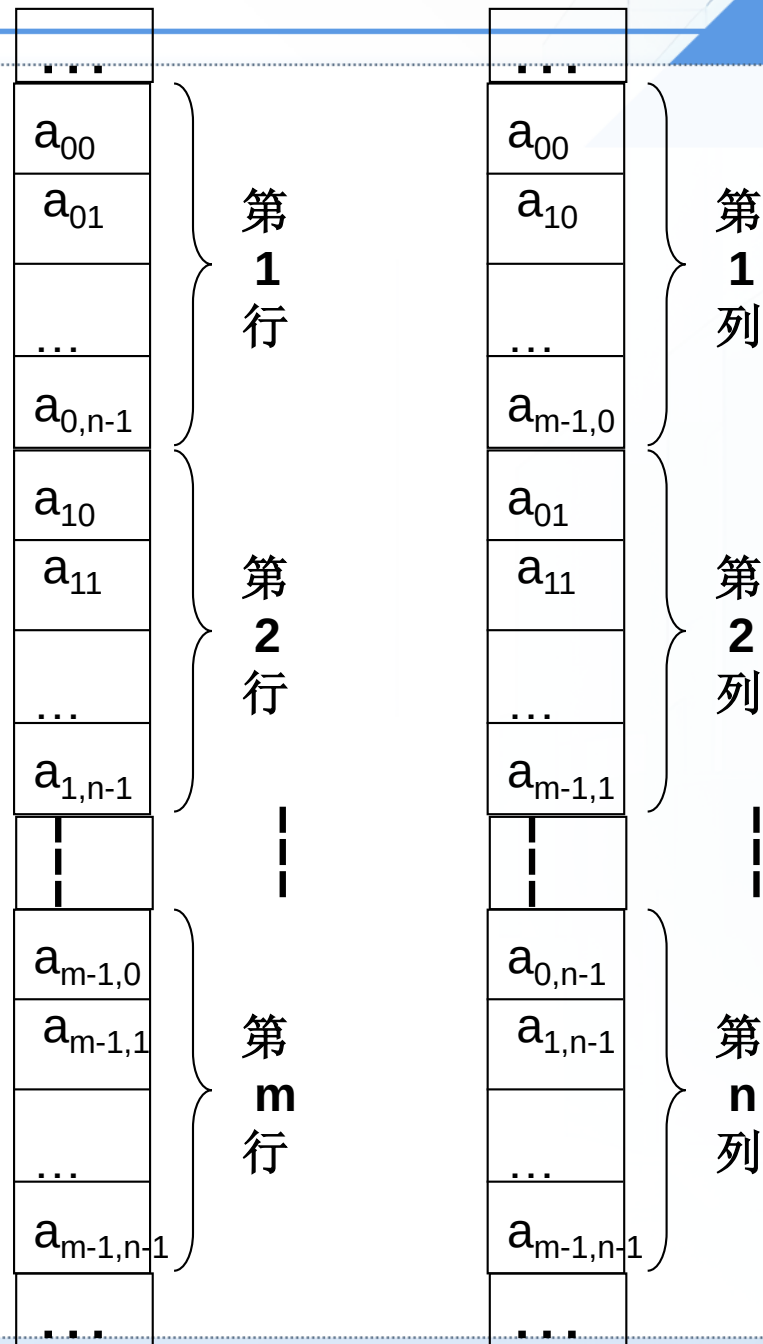
$$A = \begin{pmatrix} a_{00} & a_{01} & \dots & a_{0,n-1} \\ a_{10} & a_{11} & \dots & a_{1,n-1} \\ \dots & \dots & \dots & \dots \\ a_{m-1,0} & a_{m-1,1} & \dots & a_{m-1,n-1} \end{pmatrix}$$

(a) 二维数组的表示形式

(b) 行优先顺序存储

(c) 列优先顺序存储

二维数组及其顺序存储



设有二维数组 $A=(a_{ij})_{b_1 \times b_2}$ ，若每个元素占用的存储单元数为 L (个)， $LOC(0,0)$ 表示元素 a_{00} 的首地址，即**数组的首地址**。

1 以“**行优先顺序**”存储

二维数组中任一元素 a_{ij} 的存储地址是：

$$LOC(i,j) = LOC(0,0) + (b_2 * i + j)L \quad (5-1)$$

$$i=0,1,2,\dots,b_1-1 \quad j=0,1,2,\dots,b_2-1$$

推广到一般情况， n 维数组中任一元素 $a_{j_1 j_2 \dots j_n}$ 的存储地址是：

$$\begin{aligned} LOC(j_1, j_2, \dots, j_n) = & LOC(0, 0, \dots, 0) + (b_2 \times \dots \times b_n) \times j_1 \\ & + (b_3 \times \dots \times b_n) \times j_2 + \dots + b_n \times j_{n-1} + j_n)L \end{aligned} \quad (5-2)$$

5.3 矩阵的压缩存储

在科学与工程计算问题中，矩阵是一种常用的数学对象，在高级语言编程时，通常将一个矩阵描述为一个二维数组。这样可以对其元素进行随机存取，各种矩阵运算也非常简单。

对于**高阶矩阵**，若其中**非零元素呈某种规律分布**或者**矩阵中有大量的零元素**，若仍然用常规方法存储，可能存储重复的非零元素或零元素，将造成存储空间的大量浪费。对这类矩阵进行压缩存储：

- ◆ 多个相同的非零元素只分配一个存储空间；
- ◆ 零元素不分配空间。



5.3.1 特殊矩阵

特殊矩阵：是指非零元素或零元素的分布有一定规律的矩阵。

1 对称矩阵

若一个 n 阶方阵 $A=(a_{ij})_{n \times n}$ 中的元素满足性质：

$$a_{ij}=a_{ji} \quad 1 \leq i, j \leq n$$

则称 A 为对称矩阵，如图所示。

$$A = \begin{pmatrix} 1 & 5 & 1 & 3 & 7 \\ 5 & 0 & 8 & 0 & 0 \\ 1 & 8 & 9 & 2 & 6 \\ 3 & 0 & 2 & 5 & 1 \\ 7 & 0 & 6 & 1 & 3 \end{pmatrix}$$

$$A = \begin{pmatrix} a_{11} & & & & \\ a_{21} & a_{22} & & & \\ a_{31} & a_{32} & a_{33} & & \\ \dots & \dots & \dots & \dots & \\ a_{n1} & a_{n2} & \dots & & a_{nn} \end{pmatrix}$$

对称矩阵中的**元素关于主对角线对称**，因此，让每一对对称元素 a_{ij} 和 a_{ji} ($i \neq j$) 分配一个存储空间，则 n^2 个元素压缩存储到 $n(n+1)/2$ 个存储空间，能节约近一半的存储空间。

不失一般性，假设按“**行优先顺序**”存储下三角形(包括对角线)中的元素。

设用一维数组(向量) $sa[0 \dots n(n+1)/2]$ 存储 n 阶对称矩阵，如图所示。为了便于访问，必须找出矩阵 A 中的元素的下标值(i, j)和向量 $sa[k]$ 的下标值 k 之间的对应关系。

K	0	1	2	3	...		n(n-1)/2		...		n(n+1)/2
sa	a ₁₁	a ₂₁	a ₂₂	a ₃₁	a ₃₂	a ₃₃	...	a _{n1}	a _{n2}	...	a _{nn}

对称矩阵的压缩存储示例

若 $i \geq j$: a_{ij} 在下三角形中, 直接保存在sa中。 a_{ij} 之前的 $i-1$ 行共有元素个数: $1+2+\dots+(i-1)=i \times (i-1)/2$, 而在第 i 行上, a_{ij} 之前恰有 $j-1$ 个元素, 因此, 元素 a_{ij} 保存在向量sa中时的下标值 k 之间的对应关系是:

$$k = i \times (i-1)/2 + j - 1 \quad i \geq j$$

若 $i < j$: 则 a_{ij} 是在上三角矩阵中。因为 $a_{ij} = a_{ji}$, 在向量sa中保存的是 a_{ji} 。依上述分析可得:

$$k = j \times (j-1)/2 + i - 1 \quad i < j$$

根据上述的下标对应关系, 对于矩阵中的任意元素 a_{ij} , 均可在一维数组sa中唯一确定其位置 k ; 反之, 对所有 $k=0,1,2,\dots,n(n+1)/2-1$, 都能确定sa[k]中的元素在矩阵中的位置 (i,j) 。称sa[n(n+1)/2]为n阶对称矩阵A的压缩存储。

2 三角矩阵

以主对角线划分，三角矩阵有上三角和下三角两种。

上三角矩阵的下三角（不包括主对角线）中的元素均为常数**c**（一般为**0**）。下三角矩阵正好相反，它的主对角线上方均为常数，如图所示。

$$\begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ c & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ c & c & \dots & a_{nn} \end{pmatrix}$$

(a) 上三角矩阵示例

$$\begin{pmatrix} a_{11} & c & \dots & c \\ a_{21} & a_{22} & \dots & c \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix}$$

(b) 下三角矩阵示例



三角矩阵中的重复元素c可共享一个存储空间，其余的元素正好有 $n(n+1)/2$ 个，因此，三角矩阵可压缩存储到向量sa[n(n+1)/2]中，其中c存放在向量的第1个分量中。

上三角矩阵元素 a_{ij} 保存在向量sa中时的下标值k与(i, j)之间的对应关系是：

$$K = \begin{cases} i \times (i-1) / 2 + j - 1 & \text{当 } i \geq j \text{ 时} \\ n \times (n+1) / 2 & \text{当 } i < j \text{ 时} \end{cases} \quad 1 \leq i, j \leq n$$

下三角矩阵元素 a_{ij} 保存在向量sa中时的下标值k与(i, j)之间的对应关系是：

$$K = \begin{cases} i \times (i-1) / 2 + j - 1 & \text{当 } i \leq j \text{ 时} \\ n \times (n+1) / 2 & \text{当 } i > j \text{ 时} \end{cases} \quad 1 \leq i, j \leq n$$

3 对角矩阵

矩阵中，除了主对角线和主对角线上或下方若干条对角线上的元素之外，其余元素皆为零。即所有的非零元素集中在以主对角线为中心的带状区域中，如图所示。

$$A = \begin{pmatrix} a_{11} & a_{12} & 0 & \dots & 0 \\ a_{21} & a_{22} & a_{23} & 0 & \dots & 0 \\ 0 & a_{32} & a_{33} & a_{34} & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & \dots & 0 & a_{n-1\ n-2} & a_{n-1\ n-1} & a_{n-1\ n} \\ 0 & \dots & 0 & 0 & a_{n\ n-1} & a_{n\ n} \end{pmatrix}$$



如上图三对角矩阵，非零元素仅出现在主对角($a_{ii}, 1 \leq i \leq n$)上、主对角线上的那条对角线($a_{i,i+1}, 1 \leq i \leq n-1$)、主对角线下的那条对角线上($a_{i+1,i}, 1 \leq i \leq n-1$)。显然，当 $|i-j| > 1$ 时，元素 $a_{ij} = 0$ 。

由此可知，一个 k 对角矩阵(k 为奇数) A 是满足下述条件：
当 $|i-j| > (k-1)/2$ 时， $a_{ij} = 0$

一个 k 对角矩阵可按**行优先顺序**或**对角线顺序**，将其压缩存储到一个向量中，并且也能找到每个非零元素和向量下标的对应关系。

上述各种特殊矩阵，其非零元素的分布都是有规律的，因此总能找到一种方法将它们压缩存储到一个向量中，对矩阵的元素进行随机存取。

5.3.2 稀疏矩阵

稀疏矩阵(Sparse Matrix): 对于稀疏矩阵, 目前还没有一个确切的定义。设矩阵A是一个 $n \times m$ 的矩阵中有 s 个非零元素, 设 $\delta = s / (n \times m)$, 称 δ 为稀疏因子, 如果某一矩阵的稀疏因子 δ 满足 $\delta \leq 0.05$ 时称为稀疏矩阵, 如图所示。

$$A = \begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 0 & 0 & 4 \\ 0 & 0 & 24 & 0 & 0 & 2 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -7 & 0 \\ 0 & 0 & 0 & -6 & 0 & 0 & 0 & 0 \end{pmatrix}$$



稀疏矩阵的压缩存储

对于稀疏矩阵，采用压缩存储时，只**存储非0元素**。必须存储非0元素的行下标值、列下标值、元素值。因此，一个三元组 (i, j, a_{ij}) 唯一确定稀疏矩阵的一个非零元素。

如图的稀疏矩阵M的三元组线性表为：
 $((1,2,12), (1,3,9), (3,1,-3), (3,6,14),$
 $(4,3,24), (5,2,18), (6,1,15), (6,4,-7))$

$$M = \begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 & 0 \end{pmatrix}$$

1 三元组顺序表

若以行序为主序，稀疏矩阵中所有非0元素的三元组，就可以得构成该稀疏矩阵的一个三元组顺序表。

(1) 三元组结点定义

```
#define MAXSIZE 12500
```

```
typedef struct {
```

```
    int i, j;
```

```
    ElemType e;
```

```
} Triple ;
```

```
typedef struct {
```

```
    Triple data[MAXSIZE+1] ; //0下标不用
```

```
    int mu, nu, tu; //行数、列数、非零元个数
```

```
} TSMatrix;
```



- ◆ 矩阵的运算包括矩阵的转置、矩阵求逆、矩阵的加减、矩阵的乘除等。转置运算是最简单的矩阵运算。
- ◆ 对于一个 $m*n$ 的矩阵 M ，它的转置矩阵 T 是一个 $n*m$ 矩阵，且 $T(i,j)=M(j,i)$ $1 \leq i \leq n, 1 \leq j \leq m$ 。
- ◆ 如无特殊说明，均假定以行序为主序存储矩阵。

i	j	v
1	2	12
1	3	9
3	1	-3
3	6	14
4	3	24
5	2	18
6	1	15
6	4	-7

M.data

i	j	v
1	3	-3
1	6	15
2	1	12
2	5	18
3	1	9
3	4	24
4	6	-7
6	3	14

T.data

求转置矩阵的基本算法思想是：

- ① 将矩阵的行、列下标值交换。即将三元组表中的行、列位置值 i 、 j 相互交换（上图的前2列）；
- ② 重排三元组表中元素的顺序。即保证交换后仍然是**以行序为主序**排序的。

方法一：

算法思想：按稀疏矩阵 M 的三元组表 $M.data$ 中的**列次序**依次找到相应的三元组存入 $T.data$ 中。

每找转置后矩阵的一个三元组，需从头至尾扫描整个三元组表 $M.data$ 。找到之后自然就成为按行优先的转置矩阵的压缩存储表示。

算法5.1 , 三元组转置 :

```
Status TransposeSMatrix(TSMatrix M , TSMatrix &T) {  
    T.mu=M.nu; T.nu=M.mu; T.tu=M.tu;  
    if (T.tu) {    q=1;  
        for(col=1; col<=M.nu ; ++col)  
            for(p=1;p<=M.tu; ++p)  
                if (M.data[p].j==col) {  
                    T.data[q].i=M.data[p].j; T.data[q].j=M.data[p].i;  
                    T.data[q].e=M.data[p].e; ++q;  
                }  
    }  
    return OK;  
} //TransposeSMatrix
```

算法主要的工作是在p和col的两个循环中完成的
故算法的时间复杂度为 $O(n \times tu)$, 即矩阵的列
数和非0元素的个数的乘积成正比。

一般传统矩阵的转置算法为：

```
for(col=1; col<=nu ;+ +col)  
    for(row=0 ; row<=mu ;+ +row)  
        T[col][row]=M[row][col] ;
```

时间复杂度为 $O(\mu \times \nu)$ 。当非零元素的个数 tu 和 $\mu \times \nu$ 同数量级时，算法TransposeSMatrix的时间复杂度为 $O(\mu \times \nu^2)$ 。

由此可见，虽然节省了存储空间，但时间复杂度却大大增加。所以上述算法只适合于稀疏矩阵中非0元素的个数 tu 远远小于 $\mu \times \nu$ 的情况。

方法二（快速转置的算法）

算法思想：直接按照稀疏矩阵M的三元组表M.data的次序依次顺序转换，将转换后的三元组**直接填入**三元组表T.data的**恰当位置**。

前提：若能预先确定原矩阵M中每一列的(即T中每一行)第一个非0元素在T.data中应有的位置，则在作转置时就可直接放在T.data中恰当的位置。因此，应**先求得M中每一列的非0元素个数**。附设两个辅助向量num[]和cpot[]。

- ◆ num[col]：统计M中第col列中非0元素的个数；
- ◆ cpot[col]：指示M中第一个非0元素在T.data中的恰当位置。

显然有位置对应关系：

$$\begin{cases} \text{cpot}[1]=1 \\ \text{cpot}[\text{col}]=\text{cpot}[\text{col}-1]+\text{num}[\text{col}-1] & 2\leq\text{col}\leq\text{M.nu} \end{cases}$$

i	j	v
1	2	12
1	3	9
3	1	-3
3	6	14
4	3	24
5	2	18
6	1	15
6	4	-7

利用前述的矩阵M的三元组表可以求得
num[col]和cpot[col]的值如下表：

col	1	2	3	4	5	6	7
num[col]	2	2	2	1	0	1	0
cpot[col]	1	3	5	7	8	8	9

算法5.2，快速三元组转置：

Status FastTransposeSMatrix(TSMatrix M , TSMatrix &T) {

T.mu=M.nu; T.nu=M.mu; T.tu=M.tu;

```
if (T.tu) {
```

```
for(col=1;col<=M.nu;++col)
```

```
num[col]=0;
```

```
for(t=1;t<=M.tu;++t)
```

```
++num[M.data[t].j];
```

```
cpot[1]=1;
```

```
for(col=2;col<=M.nu;++col)
```

```
cpot[col]=cpot[col-1]+num[col-1]; //每列的起始位置
```

//这是初值，后续要自增

//接上页

```
    for(p=1;p<=M.tu;++p) {  
        col=M.data[p].j; //取对应列号  
        q=cpot[col]; //取当前列的目标位置  
        T.data[q].i=M.data[p].j;  
        T.data[q].j=M.data[p].i;  
        T.data[q].e=M.data[p].e;  
        ++cpot[col]; //又填入了一个元素，当前列的目标位置 + 1  
    } //for  
} //if  
return OK;  
} //FastTransposeSMatrix
```

算法的时间复杂度为 $O(nu+tu)$ ，在矩阵非0元素的个数 tu 与 $mu \times nu$ 相当时，算法时间复杂度与经典算法的时间复杂度相同。

2 行逻辑链接的三元组顺序表

将上述方法二中的辅助向量cpot[]固定在稀疏矩阵的三元组表中，用来指示“行”的信息，可得到另一种顺序存储结构：**行逻辑链接的三元组顺序表**。其类型描述如下：

```
#define MAXRC 100  
typedef struct {  
    Triple data[MAXSIZE+1];  
    int rpos[MAXRC+1];  
    int mu, nu, tu;  
} RLSMatrix ;
```



稀疏矩阵的乘法

设有两个矩阵： $M=(M_{ij})_{m1 \times n1}$ ， $N=(N_{ij})_{m2 \times n2}$

则当 $n1=m2$ 时，可以计算 $Q=M \times N$ ， $Q=(Q_{ij})_{m1 \times n2}$

经典算法是三重循环：

```
for( i=1 ; i<=m1 ; ++i)
    for ( j=1 ; j<=n2 ; ++j)
    {   Q[i][j]=0 ;
        for ( k=1 ; k<=n1 ; ++k)
            Q[i][j] += M[i][k] × N[k][j];
    }
```

此算法的复杂度为 $O(m1 \times n1 \times n2)$ 。

- ◆ 设有两个稀疏矩阵 $M=(M_{ij})_{m1 \times n1}$ ， $N=(N_{ij})_{m2 \times n2}$ ，其存储结构采用行逻辑链接的三元组顺序表。此时矩阵乘法就不能使用经典算法了。

i	j	v
1	1	3
1	4	5
2	2	-1
3	1	2

M.data

i	j	v
1	2	2
2	1	1
3	1	-2
3	2	4

N.data

i	j	v
1	2	6
2	1	-1
3	2	4

Q.data

上面的三元组中， $Q=M \times N$ ，那算法应该如何设计呢？

算法思想

- ① 为得到非0的乘积，只要对 $M.data[1...M.tu]$ 中每个元素 $(i, k, M_{ik})(1 \leq i \leq m_1, 1 \leq k \leq n_1)$ ，找到 $N.data$ 中所有相应的元素 $(k, j, N_{kj})(1 \leq k \leq m_2, 1 \leq j \leq n_2)$ 相乘即可。

$N.rpos[row]$ 指示了矩阵 N 的第 row 行中第一个非0元素在 $N.data[]$ 中的位置(序号)，显然， $N.rpos[row+1]-1$ 指示了第 row 行中最后一个非0元素在 $N.data[]$ 中的位置(序号)。

- ② 对于 M 中的每个元素 $M.data[p](p=1, 2, \dots, M.tu)$ ，找到 N 中所有满足条件 $M.data[p].j=N.data[q].i$ 的元素 $N.data[q]$ ，求得 $M.data[p].e \times N.data[q].e$ ，该乘积是 Q_{ij} 中的一部分。求得所有这样的乘积并累加求和就能得到 Q_{ij} 。

算法5.3 , 三元组矩阵乘法 :

```
Status MultSMatrix(RLSMatrix M, RLSMatrix N, RLSMatrix &Q)
{
```

```
    if (M.nu != N.mu) return ERROR;
```

```
    Q.mu=M.mu; Q.nu=N.nu; Q.tu=0;
```

```
    if (M.tu * N.tu != 0) { //Q是非零矩阵
```

```
        for(arow=1;arow<=M.mu;++arow) { //逐行处理
```

```
            ctemp[]=0; //当前行各列元素累加器清零
```

```
            Q.rpos[arow]=Q.tu+1;
```

```
            //tp表示M的当前行的下一行的起始位置 ( 用作循环终点 )
```

```
            if (arow<M.mu) tp=M.rpos[arow+1];
```

```
            else tp=M.tu+1;
```

//接上页

```
for(p=M.rpos[arow];p<tp;++p) {  
    brow=M.data[p].j; //找到对应元在N中的行号  
    //t表示N的当前行的下一行的起始位置（用作循环终点）  
    if (brow<N.mu) t=N.rpos[brow+1];  
    else t=N.tu+1;  
    for(q=N.rpos[brow];q<t;++q) {  
        ccol=N.data[q].j ; //乘积元素在Q中列号  
        ctemp[ccol] += M.data[p].e * N.data[q].e;  
    }//for q  
}//for p
```

//接上页

//压缩存储该行非零元

for(ccol=1;ccol<=Q.nu;++ccol)

if (ctemp[ccol]) { //非零元添加到Q中，否则跳过

if (++Q.tu > MAXSIZE) return ERROR;

Q.data[Q.tu]=(arow, ccol, ctemp[ccol]);

}//if

}//for arow

}//if

return OK;

} //MultSMatrix

- ✓ 累加器的时间复杂度为 $O(M.mu * N.nu)$ ，求Q的非零元时间复杂度为 $O(M.tu * N.tu / N.mu)$ ，压缩存储的时间复杂度为 $O(M.mu * N.nu)$ ，因此总的时间复杂度是 $O(M.mu * N.nu + M.tu * N.tu / N.mu)$ 。
- ✓ $N.tu / N.mu$ 表示矩阵N平均每列的非零元数量。

3 十字链表

对于稀疏矩阵，当非0元素的个数和位置在操作过程中变化较大时，采用链式存储结构表示比三元组的线性表更方便。

矩阵中非0元素的结点所含的域有：**行、列、值、行指针**（指向同一行的下一个非0元）、**列指针**（指向同一列的下一个非0元）。其次，十字交叉链表还有一个头结点。两种结点的结构如下：

i	j	e
right		down

(a) 结点结构

mu	nu	tu
rhead		chead

(b) 头结点结构



//十字链表结点定义

```
typedef struct OLNode {  
    int i, j;  
    ElemType e;  
    struct OLNode *right, *down;  
} OLNode, *OLink;
```

//头结点定义

```
typedef struct {  
    OLink *rhead, *chead;  
    int mu, nu, tu;  
} CrossList ;
```

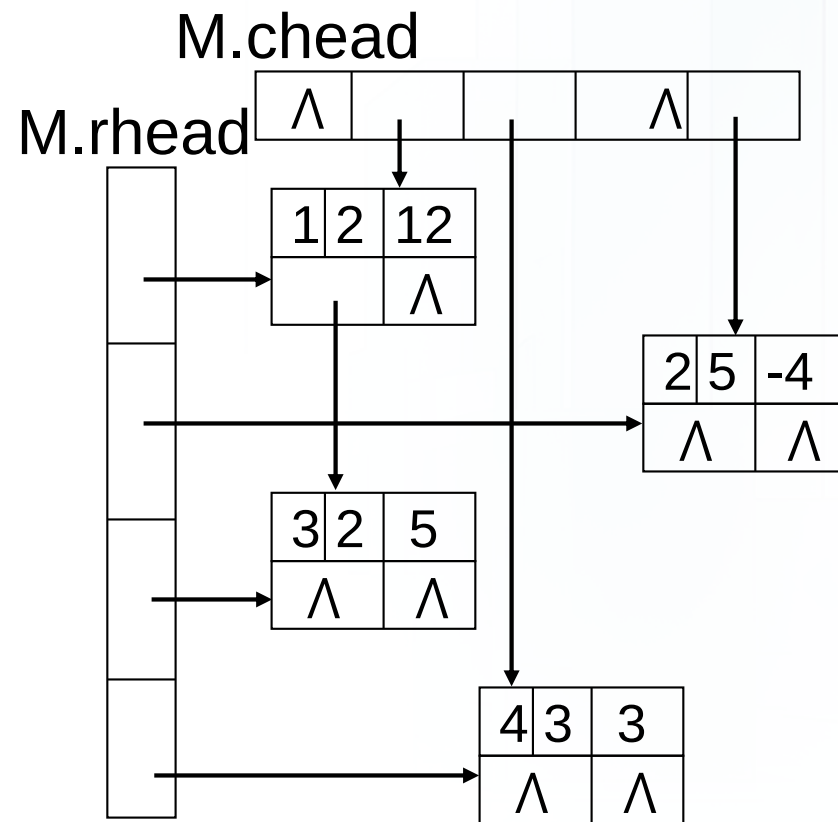
- ✓ 这里rhead和chead的类型看上去比较奇怪，它们是二级指针（OLink本身已经是一级指针）。
- ✓ 原因是，这里的rhead是作为数组首地址的。因为一个矩阵的rhead肯定有**很多个**，存储这批rhead就需要一个数组，而数组的长度就是 $\text{mu}+1$ 。chead同理。

稀疏矩阵中同一行的非0元素由right指针域链接成一个行链表，由down指针域链接成一个列链表。所有的非0元素构成一个十字交叉的链表。称为十字链表。

两个一维数组分别存储行链表的头指针和列链表的头指针。

$$M = \begin{pmatrix} 0 & 12 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -4 \\ 0 & 5 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \end{pmatrix}$$

(a) 稀疏矩阵



(b) 稀疏矩阵的十字交叉链表

算法5.4 , 创建十字链表 :

```
Status CreateSMatrix_OL(CrossList &M) {
```

```
    if (M) free(M);
```

```
    scanf(&m, &n, &t);
```

```
    M.mu=m; M.nu=n; M.tu=t;
```

```
    if (!(M.rhead=(OLink*)malloc((m+1)*sizeof(OLink))))  
        exit(OVERFLOW);
```

```
    if (!(M.chead=(OLink*)malloc((n+1)*sizeof(OLink))))  
        exit(OVERFLOW);
```

```
    M.rhead[ ]=M.chead[ ]=NULL;
```

```
    for(scanf(&i,&j,&e); i!=0; scanf(&i,&j,&e)) {
```

```
        if (!(p=(OLNode*)malloc(sizeof(OLNode)))) exit(OVERFLOW);
```

//接上页

```
p->i=i; p->j=j; p->e=e; //新结点p将被插入到两条链表中
if (M.rhead[i]==NULL || M.rhead[i]->j > j) {
    p->right=M.rhead[i];
    M.rhead[i]=p;
} else {
    for(q=M.rhead[i];q->right && q->right->j < j;
        q=q->right);
    p->right=q->right;
    q->right=p;
}
```

空链或者新结点的列号最小，插入到头结点后，并修改头结点。

定位到尾结点或者q结点的右侧结点的列号刚好超过j的地方，插入到q结点后。

//接上页

```
if (M.chead[j]==NULL || M.chead[j]->i > i) {
```

```
    p->down=M.chead[j];
```

```
    M.chead[j]=p;
```

```
} else {
```

```
    for(q=M.chead[j];q->down && q->down->i < i;
```

```
        q=q->down);
```

```
    p->down=q->down;
```

```
    q->down=p;
```

```
}
```

```
} return OK;
```

```
} //CreateSMatrix_OL
```

空链或者新结点的行号最小，插入到头结点后，并修改头结点。

定位到尾结点或者q结点的下方结点的行号刚好超过 i 的地方，插入到q结点后。

十字链表相加

假设有两个十字链表**A**和**B**，现在要计算 **$A=A+B$** 。具体思路是：

- ◆ 在相加的过程中，针对矩阵 **B** 中每一个非 0 元素，需判断提取到的 **B** 中的三元组在 **A** 相应位置上是否有非 0 元素、且相加是否 0，有三种情况：
 - ① 没有非 0 元素，此时直接加到矩阵 **A** 该行链表的对应位置上；
 - ② 有非 0 元素，且相加不为 0，此时只需更改 **A** 中对应位置上的三元组的值；
 - ③ 有非 0 元素，但相加为 0，此时需删除矩阵 **A** 中对应结点。

算法中只需要逐个提取矩阵 **B** 中的非 0 元素，然后判断矩阵 **A** 中对应位置上是否有非 0 元素，根据不同的情况相应作出处理。

//接上页

- ◆ 设指针 **pa** 和 **pb** 分别表示矩阵 **A** 和矩阵 **B** 中同一行中的结点（**pb** 和 **pa** 都是从两矩阵的第一行的第一个非 0 元素开始遍历），以上 3 种情况还可细分为以下几种场景：
 - ① 当 **pa==NULL** 或 **pa->j > pb->j** 时，此时可直接将 **pb** 结点添加到当前行（列）中，同时修改 **right** 和 **down** 指针；
 - ② 当 **pa->j < pb->j**，只需将 **pa** 指针向右推进一步；
 - ③ 当 **pa->j == pb->j**、且两结点的值相加结果不为 0，只需更改 **pa** 指向的结点的值即可；
 - ④ 当 **pa->j == pb->j**、但两结点的值相加结果为 0，就需要从矩阵 **A** 的十字链表中删除 **pa** 指向的结点。

5.4 广义表的定义

广义表是线性表的推广和扩充，在人工智能领域中应用十分广泛。

在第2章中，我们把线性表定义为 $n(n \geq 0)$ 个元素 $a_1, a_2, \dots, a_n$ 的有穷序列，该序列中的所有元素具有相同的数据类型且只能是原子项(Atom)。所谓原子项可以是一个数或一个结构，是指结构上不可再分的。若放松对元素的这种限制，容许它们具有其自身结构，就产生了广义表的概念。



广义表(Lists , 又称为列表) : 是由 $n(n \geq 0)$ 个元素组成的有穷序列 :

$$LS = (a_1, a_2, \dots, a_n)$$

其中 a_i 或者是原子项 , 或者是一个广义表。

LS是广义表的名字 , n 为它的**长度**。

若 a_i 是广义表 , 则称为LS的子表。

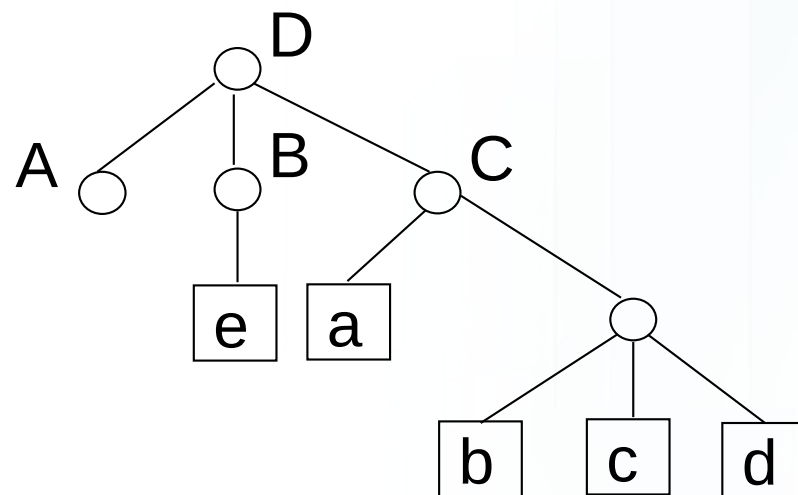
习惯上 , 原子用**小写字母** , 子表用**大写字母**。

若广义表LS非空时 :

- ◆ a_1 (表中第一个**元素**)称为**表头Head** ;
- ◆ 其余元素组成的**子表**称为**表尾Tail** ; $(a_2, a_3, \dots, a_n)$
- ◆ 广义表中所包含的元素(包括原子和子表)的个数称为表的**长度**。
- ◆ 广义表中括号的最大层数称为**表深 (度)**。

举个例子

广 义 表	表长	表深
$A = ()$	0	0
$B = (e)$	1	1
$C = (a, (b, c, d))$	2	2
$D = (A, B, C)$	3	3
$E = (a, E)$	2	∞
$F = (())$	1	2



左侧广义表**D**的图形表示

广义表的几个结论：

1. 广义表的元素可以是原子，也可以是子表，子表的元素又可以是子表，...。即广义表是一个多层次的结构。
2. 广义表可以被其它广义表所共享，也可以共享其它广义表。广义表共享其它广义表时通过表名引用。
3. 广义表本身可以是一个递归表。
4. 根据对表头、表尾的定义，任何一个非空广义表的表头可以是原子，也可以是子表，而表尾必定是广义表。

5.5 广义表的存储结构

由于广义表中的数据元素具有不同的结构，通常用**链式存储结构**表示，每个数据元素用一个结点表示。因此，广义表中就有两类结点：

- ◆ 一类是**表结点**，用来表示广义表项，由标志域，表头指针域，表尾指针域组成；
- ◆ 另一类是**原子结点**，用来表示原子项，由标志域，原子的值域组成。

只要广义表非空，都是由表头和表尾组成。即一个确定的表头和表尾就唯一确定一个广义表。这种存储结构叫**头尾链表**，它总是尝试将广义表拆成两部分存储：表头 + 表尾。



标志tag=0	原子的值
---------	------

(a) 原子结点

标志tag=1	表头指针hp	表尾指针tp
---------	--------	--------

(b) 表结点

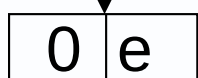
```
typedef struct { ATOM, LIST } ElemTag;
typedef struct GLNode {
    ElemTag tag;
    union {
        AtomType atom;
        struct { struct GLNode *hp , *tp ; } ptr ;
    };
} *GList;
```

hp指向表头，tp指向表尾

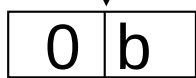
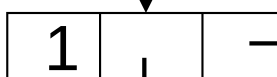
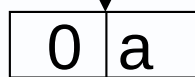
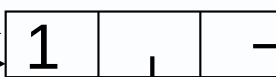
例：对 $A=()$ ， $B=(e)$ ， $C=(a, (b, c, d))$ ， $D=(A, B, C)$ ， $E=(a, E)$ 的广义表的存储结构如图所示。

$A=NULL$

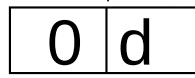
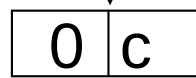
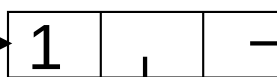
B



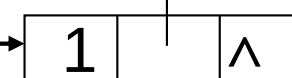
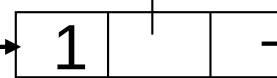
C



✓ 内层结点（包括原子结点）通过hp链接。



D



✓ 同层结点通过tp链接，例如D中，A、B、C是同层的。

◆ 对于上述存储结构，有如下几个特点：

- (1) 若广义表为空，表头指针为空；否则，表头指针总是指向一个表结点，其中hp指向广义表的表头结点（或为原子结点，或为表结点），tp指向广义表的表尾（表尾为空时，指针为空，否则必为表结点）；
- (2) 这种结构求广义表的长度、深度、表头、表尾的操作十分方便；
- (3) 最高层的表结点个数即为列表的长度。

也可用下图所示的结点结构（**同层结点链表**）存储广义表。

tag=0	原子的值	表尾指针tp
-------	------	--------

(a) 原子结点

tag=1	表头指针hp	表尾指针tp
-------	--------	--------

(b) 表结点

```
typedef struct { ATOM, LIST } ElemTag;
```

```
typedef struct GLNode {  

    ElemTag tag;  

    union {  

        AtomType atom;  

        struct GLNode *hp;  

    };  

    struct GLNode *tp;  

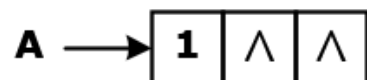
} *GList;
```

tp指向表尾，类似于next的用途

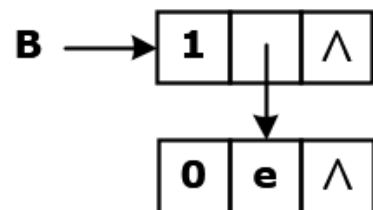
举个例子：

与头尾链表的共同点：tp链接同层结点；hp链接内层结点。

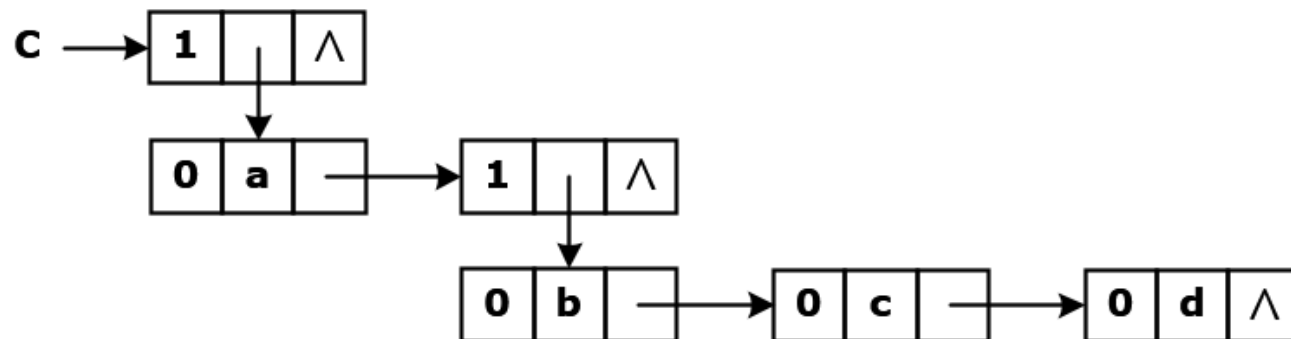
A=()



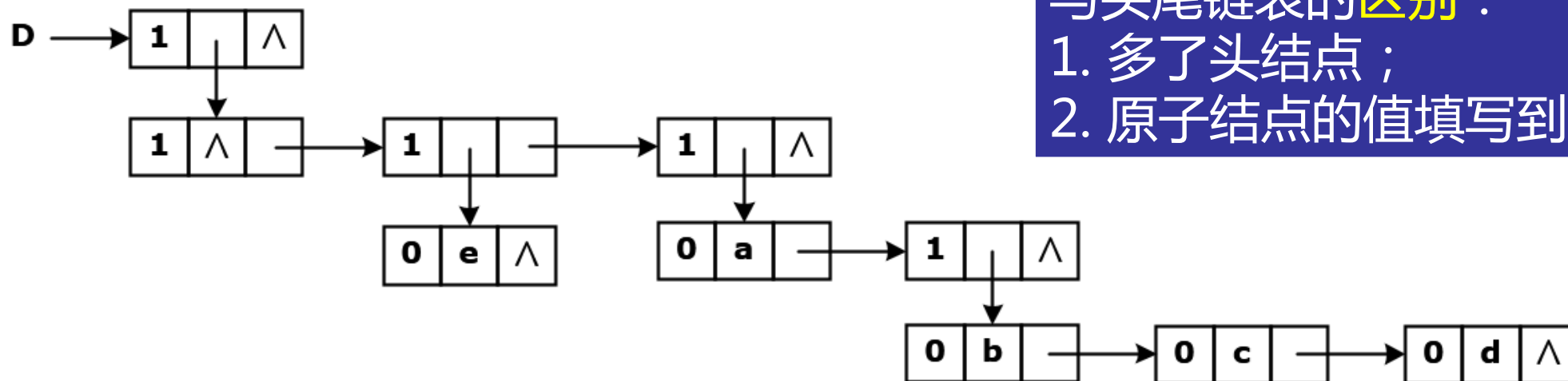
B=(e)



C=(a,(b,c,d))



D=((),(e),(a,(b,c,d)))



与头尾链表的区别：

1. 多了头结点；
2. 原子结点的值填写到中间域。

如何手搓

广义表的存储结构

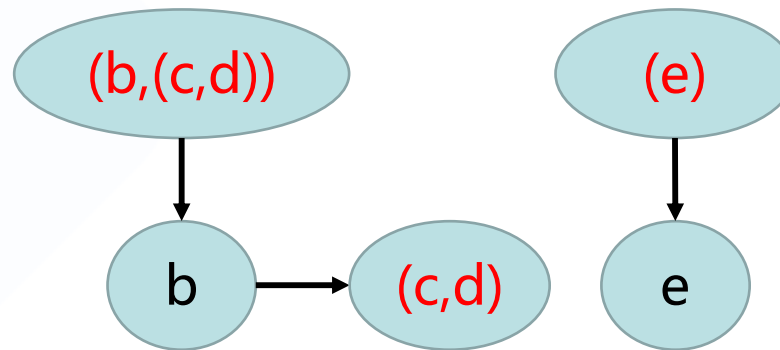


例：绘制 $E=(a, (b, (c, d)), (e))$ 的存储结构图

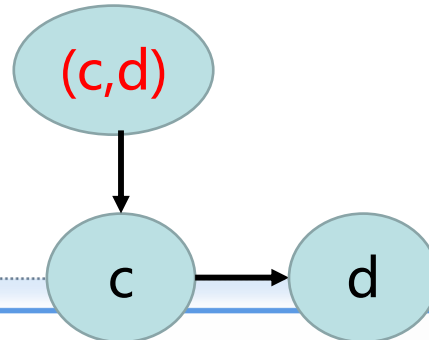
1. 先消第一层括号，将E拆分为3个子表：



2. 再消第二层括号，将后两个结点拆分子表：

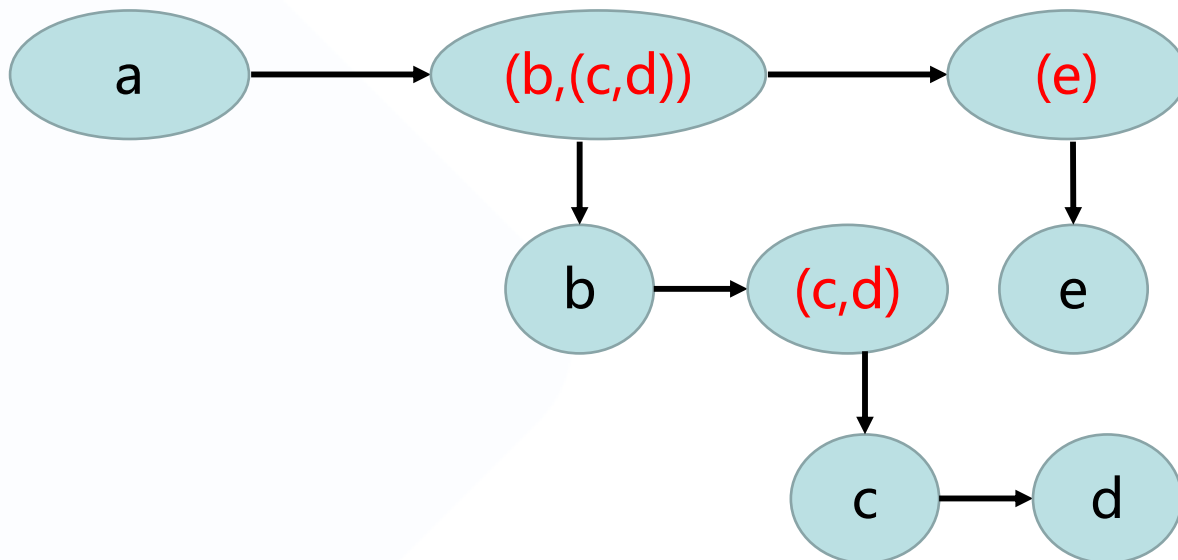


3. 再消第三层括号，将(c,d)拆分子表：



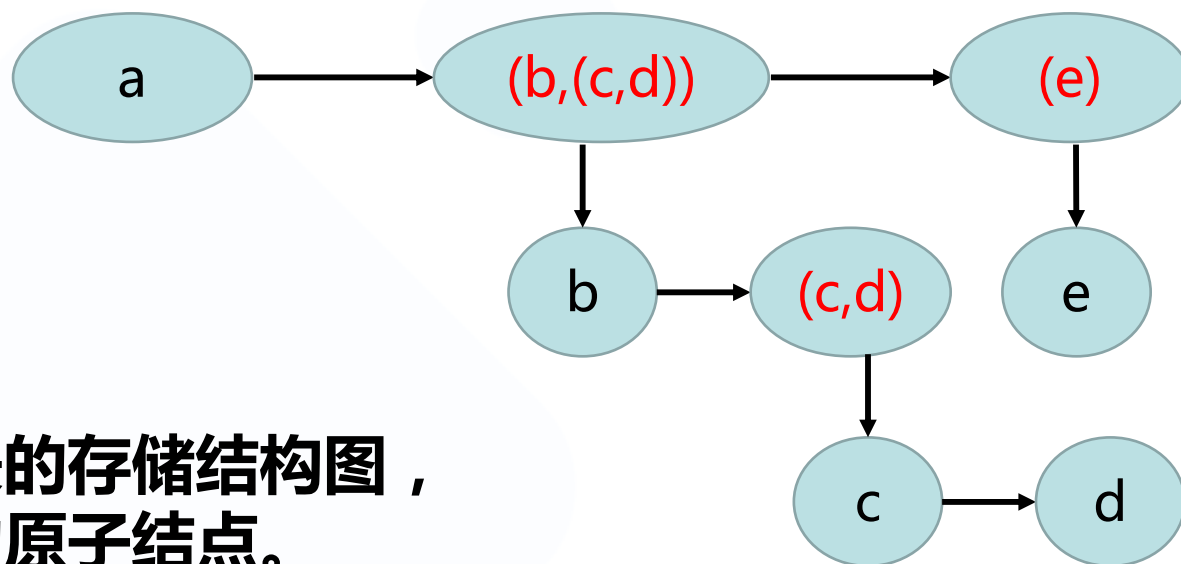
例：绘制 $E=(a, (b, (c, d)), (e))$ 的存储结构图

4. 合并以上结果，得到**逻辑结构图**（如果熟练可以直接得到这一步）：



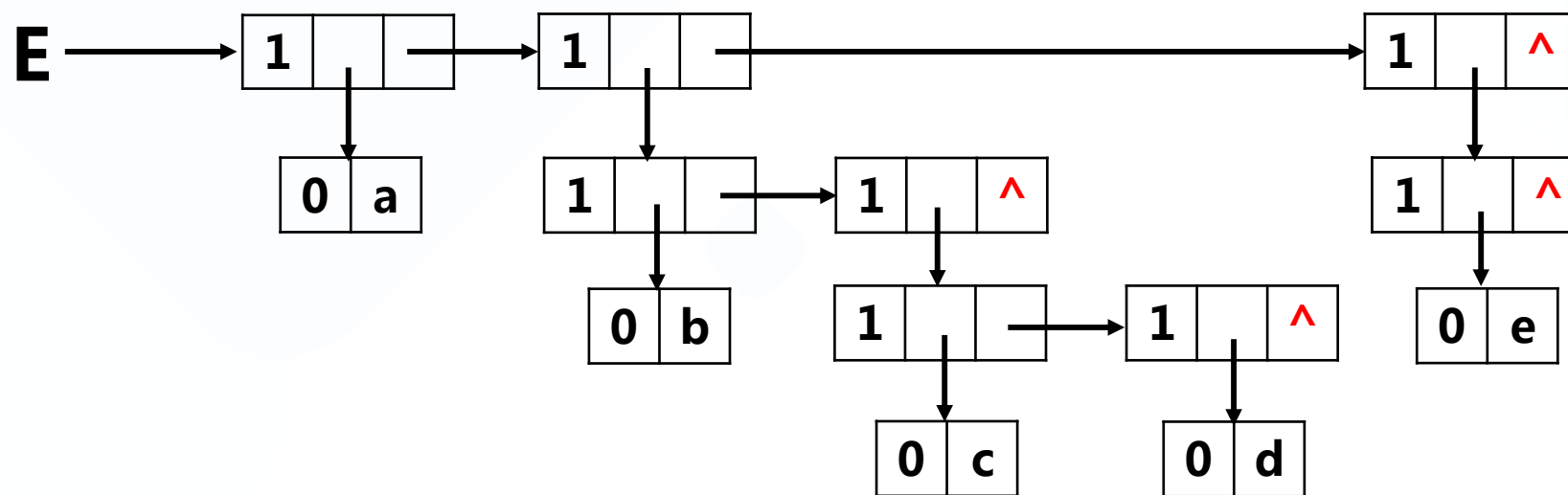
5. 注意向右的箭头是tp，向下的是hp，接下来根据广义表的存储结构（头尾链表或同层结点链表），将上述结点存储到结构体中。

例：绘制 $E=(a, (b, (c, d)), (e))$ 的存储结构图

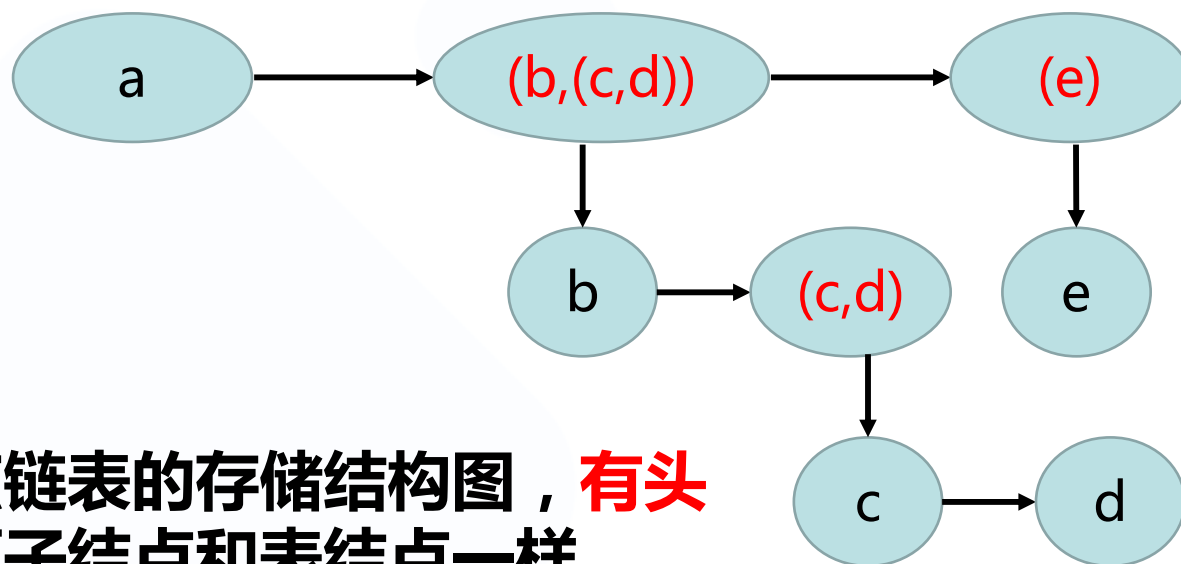


每一个原子对应两层结点，参考结点d的存储。

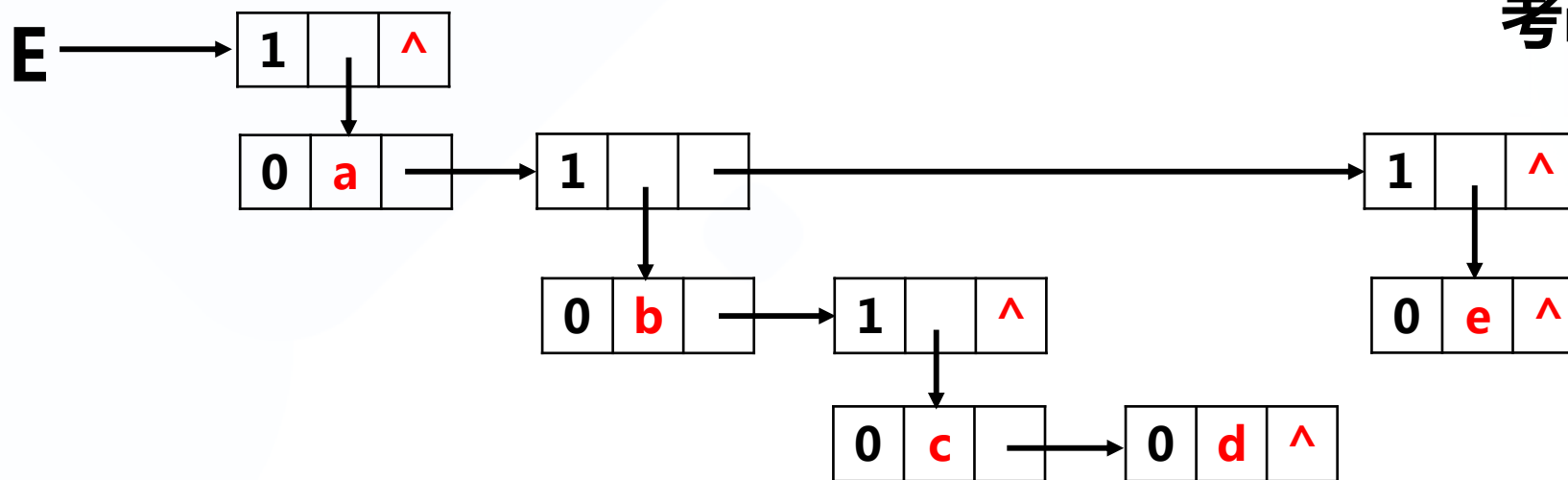
头尾链表的存储结构图，有单独的原子结点。



例：绘制 $E=(a, (b, (c, d)), (e))$ 的存储结构图



同层结点链表的存储结构图，**有头结点**，原子结点和表结点一样。



每一个原子对应一个结点。

实际使用中，**每一层都会设置头结点**，此处为了简化，只绘制了E指向的头结点。具体实例参考m元多项式。

5.6 m元多项式的表示

- ◆ 广义表很适合用来表示m元多项式。例如三元多项式表示如下：

$$P(x,y,z) = x^{10}y^3z^2 + 2x^6y^3z^2 + 3x^5y^2z^2 + x^4y^4z + 6x^3y^4z + 2yz + 15$$

然后对式子进行变形：

$$P(x,y,z) = ((x^{10} + 2x^6)y^3 + 3x^5y^2)z^2 + ((x^4 + 6x^3)y^4 + 2y)z + 15$$

- ✓ 上式可以看成是z的一元多项式。即 $P(x,y,z) = Az^2 + Bz^1 + Cz^0$
其中 $A = ((x^{10} + 2x^6)y^3 + 3x^5y^2)$ 、 $B = ((x^4 + 6x^3)y^4 + 2y)$ 、 $C = 15$ ；
- ✓ 然后二元多项式 $A(x,y)$ 又可以看成是关于y的一元多项式；B也是如此；

$$A(x,y) = ((x^{10} + 2x^6)y^3 + 3x^5y^2) = Cy^3 + Dy^2$$

其中 $C = x^{10} + 2x^6$ 、 $D = 3x^5$ 、可以看出C、D是关于x的一元多项式；

这种嵌套结构可以把m元多项式层层分解，技巧是**每次分解出一个主变元。**



◆ 把上面的几个式子排在一起：

公式	对应的广义表
$P(x,y,z)=Az^2+Bz^1+Cz^0$	$P=z((A, 2), (B, 1), (15, 0))$
$A=((x^{10}+2x^6)y^3+3x^5y^2)$	$A=y((C, 3), (D, 2))$
$B=((x^4+6x^3)y^4+2y)$	$B=y((E, 4), (F, 1))$
$C=x^{10}+2x^6$	$C=x((1, 10), (2, 6))$
$D=3x^5$	$D=x((3, 5))$
	$E=x((1, 4), (6, 3))$
	$F=x((2, 0))$

m元多项式结点定义及图示，这是典型的同层结点链表

tag=1	exp	hp	tp
-------	-----	----	----

表结点

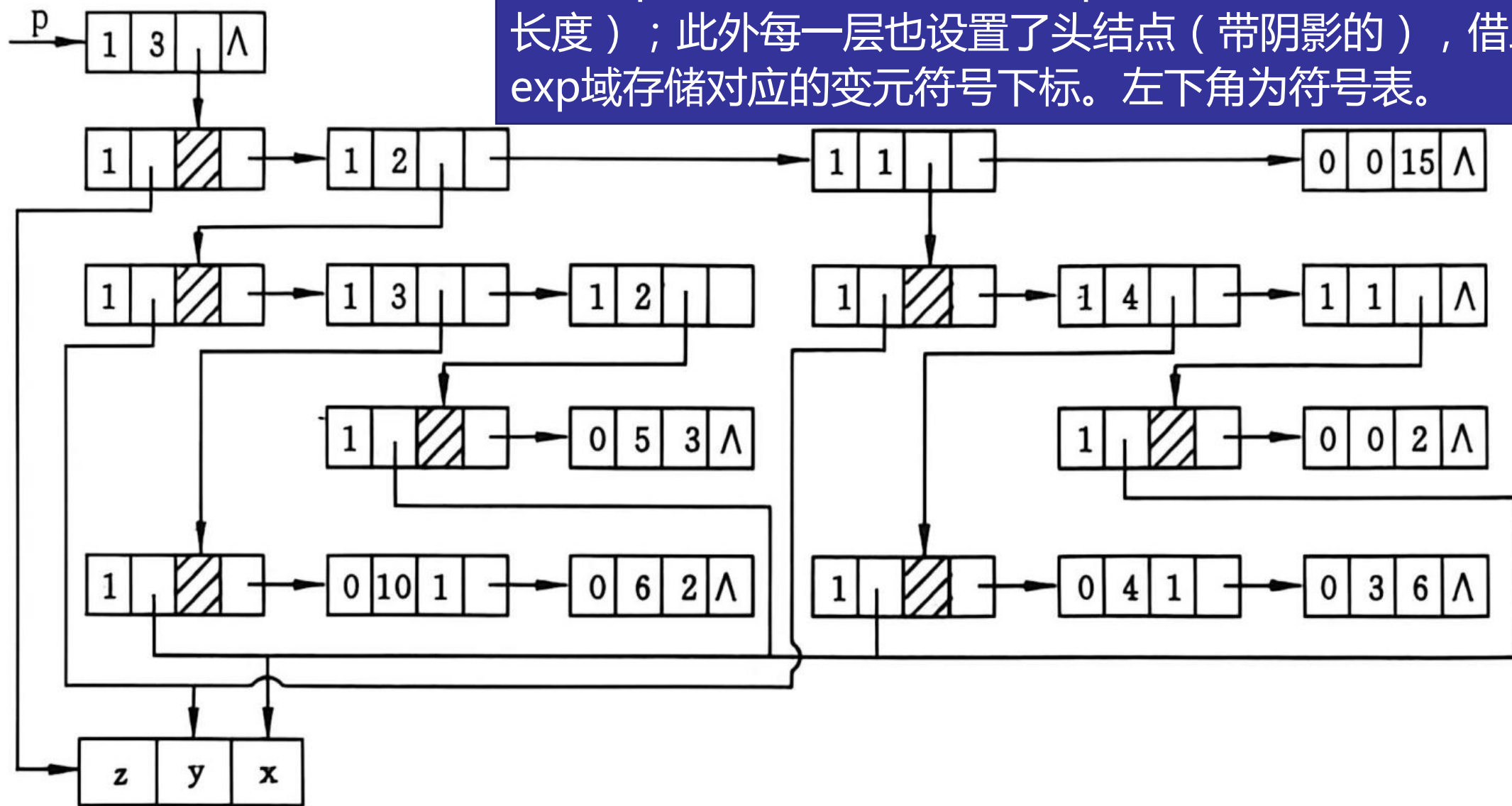
tag=0	exp	coef	tp
-------	-----	------	----

原子结点

```
typedef struct MPNode{
    ElemTag  tag; //区分原子结点、表结点
    int      exp; //指数域
    union{
        float  coef; //指向系数域
        struct MPNode *hp; //表结点的表头指针
    };
    struct MPNode *tp; //相当于线性链表的next，指向下一个元素的结点
}* MPList;
```

上面定义的广义表的存储结构示意

头指针p设置了头结点，借助exp域存储变元数量（符号表长度）；此外每一层也设置了头结点（带阴影的），借助exp域存储对应的变元符号下标。左下角为符号表。



5.7 广义表的递归算法

- ◆ 对一些可以用递归算法解决的问题，通常可以先写出问题求解的**递归定义**。和第二数学归纳类似，递归定义由**基本项**和**归纳项**两部分组成。
- ◆ 递归定义的**基本项**描述了一个或几个递归过程的**终结状态**。虽然一个有限的递归可以描述一个无限的计算过程，但任何实际应用的递归过程，除错误情况外，必定能经过有限层次的递归而终止。
- ◆ 所谓的终结状态，指的是不需要继续递归而可直接求解的状态。



- ◆ 递归定义的**归纳项**描述了如何实现从当前状态到终结状态的转化。递归设计的实质是：当一个复杂问题可以分解成若干个子问题来处理时，其中某些子问题与原问题由相同的特征属，。则可以利用与原问题相同的分析处理方法。反之，这些问题解决了，原问题也就迎刃而解了。
- ◆ 递归定义的归纳项描述的是**原问题与子问题的转化关系**。
- ◆ 递归函数的设计用的是归纳思维的方法，在设计递归函数时，应注意：严格定义函数的功能和接口；切忌想得太深太远；用归纳假设进行归纳证明时，决不能怀疑归纳假设是否正确。
- ◆ 以下针对广义表的操作，约定广义表非递归且无共享子表。



5.7.1 求广义表的深度

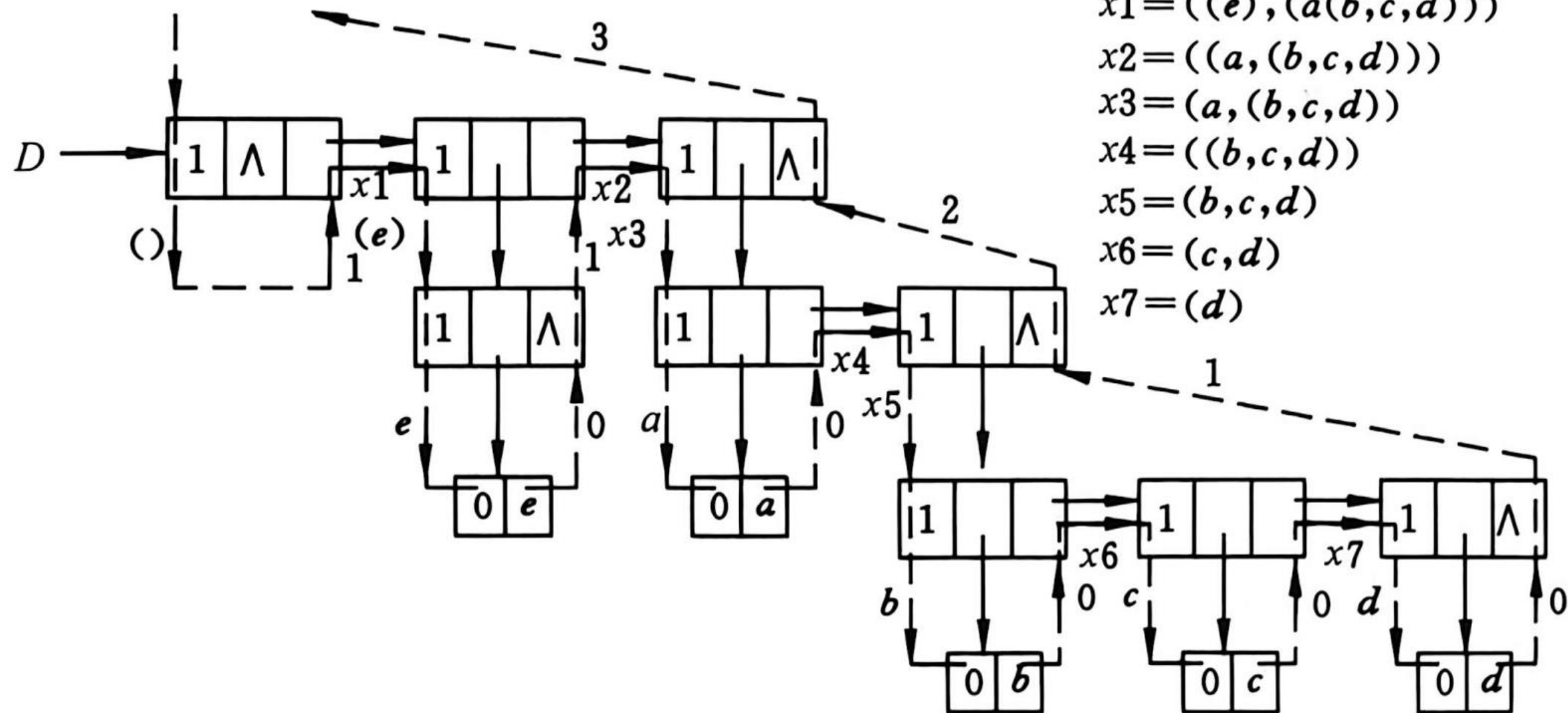
- ◆ 广义表的深度指的是广义表种括弧的重数；空表的深度为1，因为有一对括弧；原子的深度为0。
- ◆ 解题思路：
 - ① 遍历该广义表各个数据元素，求该元素的深度。如果该元素是原子，则返回深度0；如果该元素是子表，则遍历该子表的深度；
 - ② 递归的出口状态，或者叫做终结状态：当遍历数据元素为原子时返回0，当遍历数据元素为空表时，返回1；
 - ③ 设置一个变量max用来记录数据元素中最长的深度；初始化max=0；遍历过程中max与返回的整型值进行比较，取值较大的那一个。直到程序结束。max+1就是广义表的深度。

算法5.5，递归求广义表深度，这是典型的**头尾链表**：

```
int GListDepth(GList L) {  
    if (!L) return 1;  
    if (L->tag==ATOM) return 0;  
    for(int max=0, pp=L;pp;pp=pp->ptr.tp){  
        dep=GListDepth(pp->ptr.hp);  
        if (dep>max) max=dep;  
    }  
    return max+1; //非空表的深度是各元素深度最大值加1  
} //GListDepth
```

求广义表D的深度的过程

$((), (e), (a, (b, c, d)))$



5.7.2 复制广义表

- ◆ 任何一个非空的广义表均可分解成表头和表尾。反之，一对确定的表头和表尾可唯一确定一个广义表。由此，复制一个广义表只要分别复制其表头和表尾，然后合成即可。
- ◆ 所以，复制广义表的过程，其实就是不断的递归，复制广义表表头和表尾的过程。
- ◆ 递归的出口条件：
 - ◆ 如果当前遍历的数据元素为空表，则直接返回空表。
 - ◆ 如果当前遍历的数据元素为该表的一个原子，那么直接复制，返回即可。

算法5.6，递归复制广义表，这是典型的**头尾链表**：

```
Status CopyGList(GList &T, GList L){  
    if (!L) T=NULL; //直接返回空表  
    else {  
        if(!(T=(GList)malloc(sizeof(GLNode)))) exit(OVERFLOW);  
        T->tag=L->tag;  
        if(L->tag ==ATOM) T->atom=L->atom; //直接复制原子  
        else {  
            CopyGList(T->ptr.hp, L->ptr.hp); //复制表头  
            CopyGList(T->ptr.tp, L->ptr.tp); //复制表尾  
        }  
    }  
    return OK;  
}
```

5.7.3 建立广义表的存储结构

- ◆ 前面的两个操作，都是按照（表头，表尾）的方式，将一个大的广义表递归**拆分**成小的广义表。
- ◆ 其实还有一种拆分方式，就是把广义表拆分成n个并列子表，同样可以达到**拆分**表的目的。下面将以这种思路创建广义表。
- ◆ 假设要创建这个广义表： $((()),(e),(a,(b,c,d)))$
 - ◆ 首先，如果脱去最外层 $()$ ，它就是3个子表，可以逐个创建，然后再用tp指针串起来；
 - ◆ 继续上述过程，如果脱去括号后为空，或者剩下单一元素（例如e），那么就可以创建原子结点，结束递归。

算法5.7，递归创建广义表：

```
Status CreateGList(GList &L, SString S) {  
    if (StrCompare(S, emp)) L=NULL; //创建空表  
    else {  
        if (!(L=(GList)malloc(sizeof(GLNode)))) exit(OVERFLOW);  
        if (StrLength(S)==1) {  
            L->tag=ATOM; L->atom=S;  
        } //创建单原子广义表  
        else{  
            L->tag=LIST;  
            p=L;  
            SubString(sub , S , 2 , StrLength(s)-2); //脱外层()  
        }  
    }  
}
```

//接上页

```
do{ // 重复建n个子表
    sever(sub , hsub); //从sub中分离出表头串hsub
    CreateGList(p->ptr.hp, hsub); q = p;
    if (!StrEmpty(sub)) { //表尾不空
        if(!(p=(GLNode *)malloc(sizeof(GLNode))))
            exit(OVERFLOW);
        p->tag=LIST; q->ptr.tp=p;
    }
}while(!StrEmpty(sub));
q->ptr.tp=NULL;
}
} return OK;
} //CreateGList
```

算法5.8，切割字符串，hstr为第一个逗号前的子串，str为之后的：

```
Status sever(SString &str,SString &hstr){  
    n=StringLength(str);  i=0;  k=0; //k记尚未配对的左括号个数  
    do {++i;  //搜索最外层的第一个逗号  
        SubString(ch , str , i , 1);  
        if(ch=='(') ++k;  
        else if(ch==')') --k;  
    }while(i<n&&(ch!=','||k!=0));  
    if (i<n) { SubString(hstr,str,1,i-1); SubString(str,str,i+1,n-i);}  
    else { StrCopy(hstr,str); ClearString(str); }  
    return OK;  
} //server
```

结束

谢谢

